Between the Issue and the Warning
Issue Requirements and Security Warnings in Agent-Associated Pull Requests

ARNO STRAUSS, School of Computer Science and Applied Mathematics, University of the Witwatersrand, South Africa

Coding agents increasingly turn GitHub issues into pull requests (PRs), and security analysis tools report warnings on the resulting changes. A warning by itself, however, identifies neither the changed operation that produced it nor how that operation relates to what the issue requested. I examine this relationship in 3,809 merged, issue-linked JavaScript/TypeScript instances associated with coding agents through public repository indicators, 192 of which have newly recorded findings, by tracing recorded findings to the code or dependency changes responsible for them and comparing those changes with the linked issues. In the first 122 annotated instances, 605 findings map to 322 responsible changes, and seven instances hold more than half of the findings, largely through dependency advisories and repeated data-flow reports. Detailed cases show requested functionality implemented through security-relevant techniques that the issues did not specify, including biased random-character selection, regular-expression text repair and separate file checks and reads, while other findings concern tests, documentation or generated dependency changes. Two secondary analyses examine guidance. In 190 matched pairs, explicit implementation instructions were less common in finding-positive issues than in controls, although separate annotation passes limit this observational comparison. Across 12 complete retrospective replay sets, warning-informed inputs accompanied fewer recorded CodeQL alerts, but one retained output per condition and unverified functionality prevent causal conclusions. Tracing findings to changed operations and issue requirements helps interpret warnings without treating them as confirmed vulnerabilities or as evidence of the instructions an agent received.

CCS Concepts: • **Software and its engineering** → **Software creation and management**; • **Security and privacy** → *Software and application security*.

Additional Key Words and Phrases: Empirical software engineering, coding agents, GitHub issues, security analysis, pull requests, implementation provenance

2026 BSc Honours research paper. Supervised by Prof. Michael Pradel (CISPA) and Prof. Hima Vadapalli (Wits).

1 Introduction

Coding agents can now turn a plain-text request into a code change. On GitHub, an issue has traditionally been a message to a maintainer, in which a user describes a defect or a wanted feature and a person decides how to implement it. Repository-capable coding agents read the same text as a task, search the repository, edit code and open pull requests (PRs) [6, 18, 20, 23], and such PRs now appear in large numbers in open-source projects [8, 19]. Security analysis runs on this work as well. In a dataset of PRs that public signals attribute to coding agents, GitHub's rule-based security bot is among the ten most frequent bot reviewers [8]. Public issues can therefore supply instructions for automated code changes, and security analysis tools then report on the result.

That channel has a known weakness. Controlled studies show that crafted issue text can steer repair agents toward insecure or vulnerable patches that still pass tests, and that static analysis can miss them [2, 11]; prompt-injection benchmarks show that tool output and retrieved content can hijack agents, including coding editors [3, 9, 22]. These studies construct the attack, so they know which operation is responsible and what the issue was meant to cause. In naturally occurring work,

Author's Contact Information: Arno Strauss, School of Computer Science and Applied Mathematics, University of the Witwatersrand, Johannesburg, South Africa, 2613224@students.wits.ac.za.

the intended effect of an issue is harder to establish. A public issue–PR link is a starting point, not a record of every instruction or source the implementer saw, and links take several forms, including issues opened after the PR they are linked to [4].

The public record does, however, expose both the merged code change and the findings that security analysis tools report on it. What it does not show directly is how a reported finding relates to the work requested in the linked issue. A finding might concern code that directly implements the request, a test written to accompany it, or a dependency version resolved by tooling along the way, and these situations call for different interpretations. Existing work approaches the question from either side. Issue-resolution benchmarks primarily measure task success [6], and studies of coding-agent PRs describe their adoption and development patterns [8, 19]. Security studies show that a patch can pass functional checks while introducing a weakness [2, 10], and the closest work relates security findings to patch source and issue-level information [13] or to weakness categories and review outcomes in security-related PRs [12]. This study takes a complementary unit of analysis. For each recorded finding, it identifies the particular changed operation responsible and asks what the linked issue required of that operation.

A concrete case shows why this relationship matters. An issue in the teamspeak-music-bot project asks for Spotify as an audio source. The linked pull request delivers it, and in doing so builds a login verifier by turning each random byte into a character with a remainder operation over a 66-character alphabet. Because 256 byte values do not divide evenly into 66, some characters come up slightly more often than others, and CodeQL reports a biased-random warning. CodeQL identifies the flagged location, but the report alone does not establish which PR change introduced the condition or how that change relates to the request. The issue asked for Spotify support, not for this way of choosing characters.

The analysis covers 3,809 merged, issue-linked, agent-associated JavaScript/TypeScript instances, of which 192 have newly introduced scanner findings, and draws on public issue discussions, PR changes and tool output.

The central distinction is between what an issue requests and how a change implements it. A change can directly implement functionality described in an issue and still trigger a security warning when the warning concerns a technique that the issue left open. Interpreting the warning therefore requires information that the warning itself does not carry: the changed operation, its role in the requested task, and whether it lies in production code, tests, documentation or dependency files. Tracing these relationships makes a warning more informative than its presence or count alone. Figure 1 places the study in the issue-driven development workflow, and Figure 2 shows the chain from issue to finding that the research questions examine.

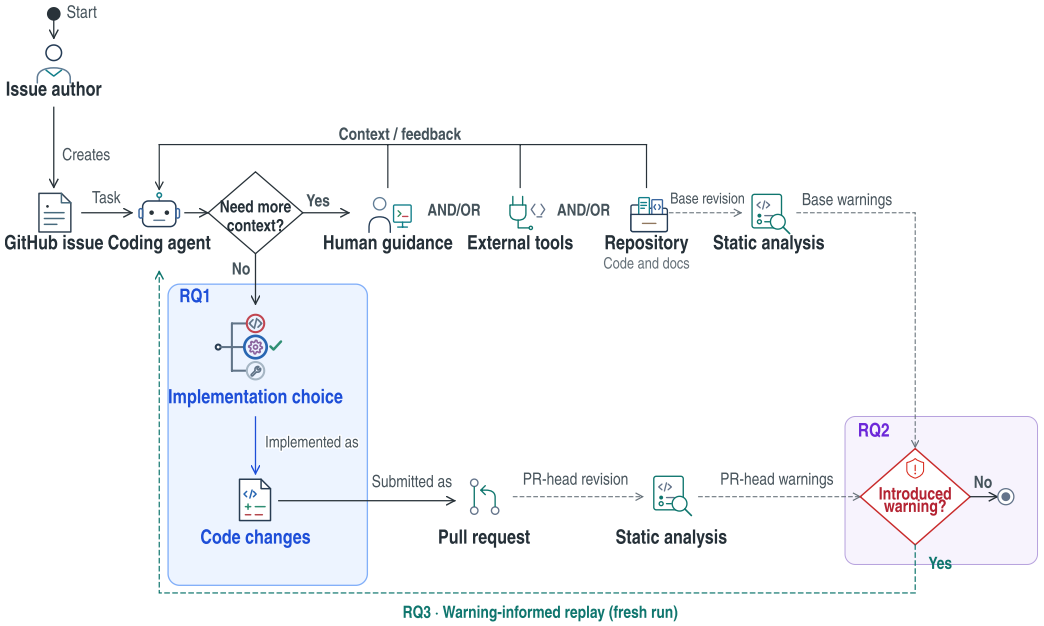


Fig. 1. Workflow overview showing issue-driven implementation, possible context sources, base/head checks and warning-informed reimplementation. The RQ2 marker denotes the new-finding comparison used to define the matched groups, rather than the wording analysis itself. This is a conceptual workflow, not a verified execution trace.

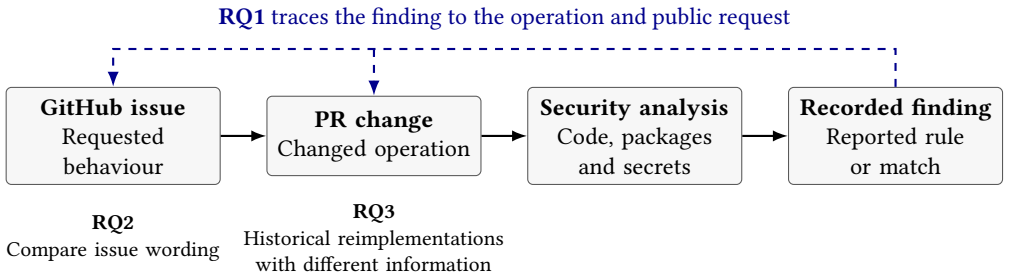


Fig. 2. The relationship examined in the study. Solid arrows show conceptual links, not verified instruction flow in an agent session. RQ1 relates a reported finding to the changed operation and the public issue. RQ2 compares issue wording, while RQ3 examines new implementations of selected historical tasks.

1.1 Research Questions

The study asks three questions of unequal weight. RQ1 provides the main evidence: it relates recorded security findings to the changed operations behind them and to the work requested in the linked issues. RQ2 and RQ3 examine related questions about guidance, namely whether issue instruction specificity differs between issues with and without new findings, and how implementations and CodeQL results differ when historical tasks are replayed with additional security information.

- **RQ1.** How do recorded security findings in agent-associated pull requests relate to what their linked issues request?
- **RQ2.** Does issue instruction specificity differ between issues linked to PRs with newly introduced findings and issues linked to matched PRs without new findings under comparable completed checks?
- **RQ3.** In exploratory replays of historical tasks, how do the implementation and CodeQL findings differ between the original issue alone, the issue with its historical warning, and the issue with a security property derived from that warning?

1.2 Contributions

The main contribution is an empirical account, at the level of individual changed operations, of how recorded security findings relate to issue requirements in naturally occurring merged PRs. The initial 122-instance analysis links 605 findings to 322 responsible changes and distinguishes changes that directly implement the requested work from changes that support it (Sections 4.3 and 4.4). Source-linked cases then show requested functionality implemented through flagged techniques that the issues did not prescribe, and an extended analysis groups 455 changes into 342 implementation choices and examines earlier public evidence for those choices. Two secondary analyses, a matched comparison of issue wording and a set of retrospective replays, explore how guidance relates to recorded findings.

2 Background

2.1 Tasks, Patches and Coding Agents

GitHub issues record requested functionality, defects and discussion; PRs record proposed changes. Their links permit analysing a request alongside a patch, but do not prove that the public issue initiated, fully specified or dictated the implementation [4]. Coding agents may also use repository instructions, files and tool output absent from the public issue [18, 20]. *Agent-associated* means that public PR metadata, branch patterns or issue/PR text indicate coding-agent involvement; a tool mention does not prove agent authorship, so the sample is not a census of autonomously written patches.

2.2 Warnings, Advisories and Retained Findings

CodeQL and Semgrep flag patterns in source code; OSV and Trivy report vulnerable dependencies, while Gitleaks identifies possible secrets. CodeQL's query-based approach builds on QL, a language designed for querying program representations [1]. Throughout the paper, a *finding* is one retained report from any of these tools, not a confirmed vulnerability. For RQ3, the recorded count is specifically the number of new CodeQL alerts in a generated patch.

A PR is *finding-positive* when the recorded before/after comparison detects at least one new finding. A *control* has no new findings under the applicable checks that completed successfully.

Controls may still have older findings or lack results from other tools; neither label establishes exploitability or functional correctness.

2.3 Task Relevance and Mechanism Choice

Task role describes how a change relates to the requested behaviour; whether the issue specified the technique used is a separate question. Suppose an issue asks for an @-mentioned file to be included in a prompt. Reading that file directly addresses the request. Checking its metadata and then reading it by pathname is one particular way of doing so, which the issue need not prescribe. The first observation concerns the task role and the second the implementation choice, and the study records them separately.

3 Related Work

3.1 Coding Agents and How They Are Judged

SWE-bench evaluates software-repair patches against tests for 2,294 GitHub issues drawn from 12 Python repositories [6]. SWE-agent, AutoCodeRover and OpenHands provide repository-level interfaces for agents to inspect files, edit code and run tests [18, 20, 23]. Trae Agent studies test-time scaling for repository-level issue resolution by generating and selecting candidate solutions [5]. These systems and benchmarks primarily evaluate task completion; the present study instead examines the relationship between recorded security findings, the changes that produced them and the issue requirements.

3.2 Agent Pull Requests and Issue Mining

Public signals such as account identities, branch names and co-authorship metadata have been used to study coding-agent PRs at scale. Li et al. analyse agent participation in open-source development, while Watanabe et al. examine how agent-associated PRs are used and integrated on GitHub [8, 19]. The links between issues and PRs have their own structure and do not necessarily establish which public text initiated a change [4]. Prior issue research examines the requests made in open-source AI projects and the characteristics of beginner-friendly issues [16, 21]. This study uses issue links and coding-agent indicators to select its cohort, then inspects the code behind recorded findings rather than treating a link as proof of the instructions received by an agent.

3.3 Crafted Issues and Untrusted Context

Przymus et al. show that adversarial bug reports can lead automated repair agents to introduce attacker-aligned changes [11]. SWExploit constructs misleading issue reports intended to induce functionally correct but vulnerable patches [2]. AgentDojo and InjecAgent benchmark prompt injection through untrusted tool outputs and other external content; Liu et al. examine attacks against coding editors [3, 9, 22]. Related defences study detecting and removing injected instructions (PromptArmor) and applying program analysis to agent runtime traces (AgentArmor) [15, 17]. These experiments investigate adversarial instructions or runtime defences. This paper instead examines naturally occurring merged PRs without assuming that their linked issues are attacks.

3.4 Static-Analysis Warnings and Generated Code

Static-analysis findings need interpretation. Johnson et al. report that developers' use of static-analysis tools is affected by false positives and how the warnings fit their workflow [7]. Pearce et al. assess the security of GitHub Copilot suggestions, showing why plausible code needs security assessment [10].

Closer work examines complete patches. Sajadi et al. compare developer, LLM and agent patches on SWE-bench using CodeQL, Semgrep and Bandit. They also examine issue-level information, including code snippets, expected behaviour and reproduction steps [13]. Their analysis therefore already connects issue context with security outcomes. Rabbi et al. apply before/after Semgrep scans to 675 agent-associated PRs selected as security-related and examine weakness categories, review outcomes and rejection reasons [12]. The present study differs in selection and in unit of analysis. It selects merged, issue-linked JavaScript/TypeScript work without requiring a security-related description, and it examines how the specific operation associated with each finding relates to the public request.

3.5 Research Gap

Existing studies evaluate task completion [6, 20], characterise agent-associated PRs [8, 19], investigate crafted attacks [2, 11], and analyse security findings in generated patches or security-related PRs [12, 13]. Together they establish that agent-associated changes can carry security findings and, in the case of Sajadi et al., that issue context can be connected with security outcomes [13]. This study's focus is the step in between. A scanner reports a finding at a location, but that location need not be code the PR changed. A data-flow report can point to a statement that predates the PR, and a dependency advisory names a package version rather than an edit. The operation responsible therefore has to be identified from the PR diff together with the scanner evidence, and only then can it be compared with what the issue asked for. Without these two steps, a finding in code that fulfils the request cannot be distinguished from one in an accompanying test or a regenerated lockfile, and a technique that the issue prescribed cannot be distinguished from one that it left open. The case analysis follows these steps in naturally occurring merged work, where, unlike in constructed attacks, neither the responsible operation nor the intended effect of the issue is known in advance. The matched comparison and retrospective replays provide supporting, exploratory evidence about issue specificity and additional security information.

4 Study Design and Methods

I conducted an observational study of merged, agent-associated PRs linked to public GitHub issues in JavaScript and TypeScript repositories. The study selected issue–PR instances, compared scanner results across each PR, linked new findings to responsible changes (RQ1), compared issue wording with matched controls (RQ2), and examined retrospective warning-informed replays (RQ3). Figure 2 shows the conceptual relationship between the public request, implementation and finding; Table 1 identifies the populations used in each analysis.

Four units recur throughout. One *instance* is a linked issue–PR pair. A *finding* is a retained scanner report, and a *responsible change* is the code or dependency operation linked to one or more findings. Related responsible changes within an instance are grouped into an *implementation choice*, which describes an observable implementation technique and is neither a separate finding nor a verified private decision by an agent. RQ2 analyses matched instance pairs, and RQ3 analyses generated patches for selected historical tasks. Table 1 keeps these overlapping populations separate.

Table 1. Study populations and units. The subsets overlap and must not be added together.

Population	Size	Use in the paper
Full corpus	3,809 instances	Merged, issue-linked PRs; scanner comparisons.
Finding-positive	192 instances	At least one newly recorded finding.
Initial annotations	122 instances	605 findings linked to 322 responsible changes.
Extended mapped set	191 instances	455 responsible changes grouped into 342 implementation choices; one positive has no mapped change.
Matched comparison	190 pairs	One positive and one control per RQ2 pair.
O/W/P replays	14 tasks	12 complete sets of three generated patches.

4.1 Population and Selection

I searched GitHub for merged PRs linked to issues (qualifiers `is:pr`, `is:merged`, `linked:issue`) in non-archived JavaScript/TypeScript repositories with at least 100 stars, merged between 1 November 2025 and 11 August 2026. JavaScript and TypeScript match the CodeQL and Semgrep rule sets applied, the start date keeps the sample recent, and the star cutoff limits the sample by repository popularity. The snapshot contains 3,809 distinct issue–PR instances in 723 repositories (609 JavaScript, 3,200 TypeScript; selection criteria and cohort characteristics in Table S1). Agent indicators include PR authorship or metadata, branch and description markers, and weaker issue-side references to Codex, Copilot, Cursor, Claude or Devin; for 236 instances the only association is an issue-side marker, not PR-side evidence. The 100-star threshold is a sampling filter rather than a quality measure. Stars and language are snapshot metadata, not values reconstructed at each merge date. The cohort is neither a random GitHub sample nor an estimate of finding prevalence among all coding-agent work.

4.2 Detecting and Retaining Introduced Findings

The pipeline records whether each scanner applies, whether it ran successfully and what it reported (Table S2). CodeQL and Semgrep findings are counted as introduced when the archived comparison reports them after, but not before, a PR. For the original cohort, retained source-code findings also have a primary or data-flow location on a PR-added line. These are *introduced reports*, not independently established new vulnerabilities. A changed data-flow path may affect an existing flagged statement (Table S2).

Dependency results received an additional merge-base verification in the first annotation cohort. Package/version records were compared at the PR's true merge base and head; this verification excluded 12 initially positive instances after four earlier exclusions. Section S2 reports the checks and their results. Equivalent verification is not established for all 70 subsequently annotated positives, so I do not generalise the stronger verification claim to them.

Of the 3,809 instances, 192 had newly recorded findings. The remaining 3,617 had none under their completed applicable checks; this does not imply that every scanner ran. Detailed annotation covered 122 positives first and 70 later. One positive had no exactly mapped change, leaving the 191-instance mapped set in Table 1. The original 122-instance batch was not a random sample. Its 605 findings and 322 responsible changes should not be confused with the 455 changes in the extended mapped set.

4.3 Removing Duplicates and Linking Findings to Changes

A finding and its responsible change are different. In T3Code, for example, two distinct CodeQL rules concern changes made as part of one regular-expression strategy for recognising scoped

packages (Section S3). Repeated records of the same finding within an instance were merged, while distinct rules and advisories were kept separate. This yielded 605 findings from 832 raw tool references in the 122-instance cohort. For source-code warnings the comparison used reported rules and locations, changed hunks and available pre-change code; for dependency findings it used declared or resolved versions before and after the PR. A reported file location alone did not establish the responsible operation; Table S4 gives the mapping basis of every finding.

I used model-assisted annotation to propose the code, configuration or dependency change responsible for each retained finding. The model received the scanner finding, tool output and PR diff, but not the issue text or PR description. A candidate link was retained only when the record identified a changed file, a line or diff hunk, and an explanation of how that change accounts for the finding. This records supporting evidence for each link, but does not constitute independent human verification of every link.

For 142 dependency findings in the original cohort, recorded merge-base verification supports the package/version attribution; 36 of these links were resolved during that verification. After exclusions, the 122-case workbook records 605 retained findings mapped to 322 changes (Table S4). The underlying dependency verification records and counts are given in Section S2.

The recorded annotation model was GPT-5.6 Sol with medium reasoning effort. Because one responsible change can account for multiple findings, the two counts are reported separately. Independent validation limits are discussed in Section 7. Subsequent task-role coding compares the mapped change with the issue's requested goals; it does not independently revalidate the finding-to-change link (Section 4.4). Case evidence is presented in Sections S3 and S6.

Figure 3 shows how retained findings are linked to responsible changes, assigned task roles and grouped into implementation choices. Model-assisted grouping of the 455 changes yields 342 choices within instances. Each choice is classified as publicly prescribed, mechanically generated, selected among alternatives or unresolved, using only evidence available before the change (Table 2). Choices made among alternatives are further described using the evidence tiers in Table S3.

In Case 3 (Section 5.2), CodeQL's file-system-race report is the finding, and the PR's added check-then-read sequence is the responsible change. The read directly addresses the requested task, although the issue does not prescribe that sequence. These two observations receive separate annotations.

4.4 Task Role, Affected Files and Earlier Public Evidence

I used model-assisted coding to assign task roles by comparing a factual operation summary with the issue's requested goals (Table 2). The labels (Direct, Supporting, Unclear) describe the operation's role, not whether its mechanism was publicly prescribed or who chose it. The annotations also record the finding family and the affected file type, distinguishing production source, tests or fixtures, documentation or examples, dependency manifests and generated lockfiles. File type refers to the finding's reported location, which need not be the file containing the mapped change. A pattern in a test or README does not imply the exposure of the same pattern in deployed code.

For the original 322 changes, an initial model-assisted pass assigned task roles using the previously fixed issue request and the change evidence, without PR-description text or complete finding records. A subsequent source-assisted quality-control pass revised 11 roles (seven Direct to Supporting and four Supporting to Direct); the workbook retains the earlier and final labels and case-specific reasons. These corrections do not constitute a second-annotator agreement study or independent revalidation of all links. The 133 later changes also have model-assisted roles.

The extended mapped set contains 455 responsible changes grouped into 342 implementation choices. Model-assisted coding then distinguishes techniques prescribed in earlier public evidence, changes produced mechanically by tools or dependency resolution, and techniques selected among

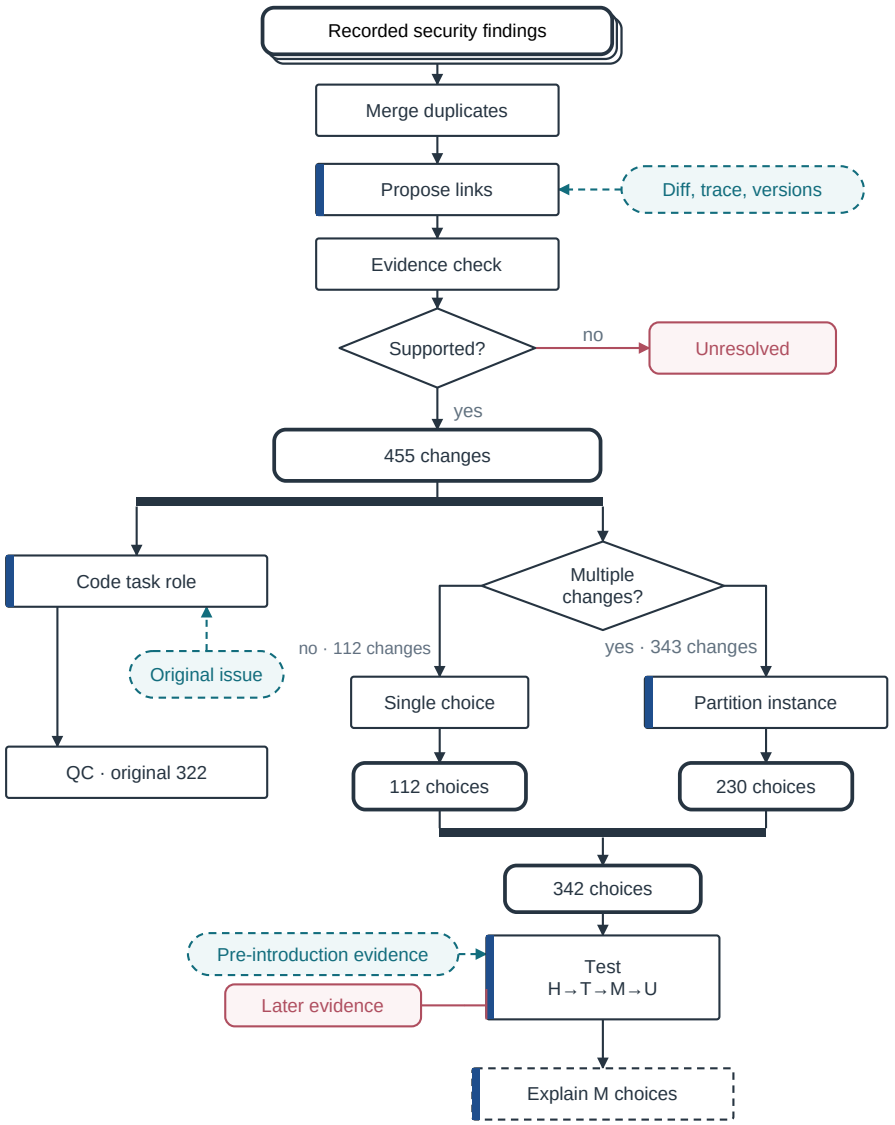


Fig. 3. From findings to responsible changes and implementation choices. Blue bars mark model-assisted steps; teal callouts identify admissible evidence, and red marks unsupported links or later evidence excluded from origin coding. Dark outlines show retained counts. The first yes/no decision tests link support; the second distinguishes changes grouped into choices. The dashed final box denotes explanation tiers for M choices. Retained links require a changed file, line or hunk and a supporting explanation (Section 4.3).

plausible alternatives. Unresolved is allowed where evidence is insufficient (Table 2). For choices made among alternatives, a separate classification describes the strength of any earlier public precedent. The primary counts use one retained classification version; the comparison with an earlier version is confined to Table S6. Independent blind human agreement was not measured.

Table 2. RQ1 classification rules and their limits. Task role is decided per responsible change; how it arose per grouped choice.

Category	Decision rule	Interpretation
Task role (per responsible change)		
Direct	Implements a substantive goal of the public issue.	The issue need not prescribe the mechanism.
Supporting	Enables or accompanies the requested work rather than fulfilling it, including incidental implementation choices the issue did not request.	Role is not inferred from file type alone.
Unclear	Evidence cannot establish the task relation.	Uncertainty is retained, not forced into a binary.
How the implementation choice arose (per grouped choice)		
H — prescribed	Pre-introduction public evidence specifically fixes the mechanism.	A broad goal or later evidence is insufficient.
T — mechanical	Dependency resolution, generated artifacts or tooling produce the choice.	Not evidence of discretionary choice.
M — selected among alternatives	One implementable technique was used although public evidence did not require it and plausible alternatives exist.	A known precedent may explain the choice; its source is not established.
U — unresolved	Evidence cannot separate H, T and M.	Missing evidence is not proof of choice.
Earlier public evidence	For M choices, note any reused code, repository precedent, stated contract or contextual support.	Its availability does not show that the contributor saw it.

I assessed earlier public evidence against the recorded time of the change. A pre-existing helper or discussion is a possible source, not evidence that the contributor viewed it. Some linked issues were created after their PRs. In the original 122 instances, 4 PRs were opened before their linked issue and 15 more within 10 minutes after it. These are PR creation times, not the time the implementing code first appeared, and the public issue may omit private user or agent messages. A Direct label therefore records that a change addresses the public issue's goals; it does not establish that the issue preceded or prompted the implementation. A technique's first appearance in the patch, or the absence of an earlier public example, does not establish that the agent invented it.

4.5 Matched Controls and Issue Coding

RQ2 compares how issues are worded in finding-positive instances and in matched controls with no newly introduced finding under the checks that applied and completed. Controls may carry older findings, and not every tool ran on each instance. The recorded matching procedure paired each PR with new findings with at most one control PR without recorded new findings. Issue-wording annotations were joined after matching. Eligible controls had to cover the positive instance's completed applicable checks, fall within 90 days of its PR creation date and have changed-line and changed-file counts within a factor of about four. Cross-repository controls required exactly the same set of completed checks. The matcher preferred controls from the same repository. Issue wording, review variables and agent identity were not additional matching criteria (Figure S2; protocol in Section S5). Of the 192 positives, 190 were matched (115 same-repository, 75 cross-repository; Table S7). The analysis used the existing matching records without rerunning the procedure. Matching does not equate the tasks or randomise the wording.

Each issue received one of three wording labels, Problem only, Suggests solution or Explicit instruction. They describe the specificity of wording, not whether a security requirement was stated or the exact mechanism prescribed. An Explicit instruction label therefore does not imply an H classification for the implementation choice. Positive issues and controls were coded in separate passes; the control pass withheld scanner outcomes and pair identities. Because cross-pass agreement was not measured, some of the wording difference could reflect coding differences (Section 7). The 121 original-cohort pairs and the 115 same-repository pairs are nested sensitivity subsets, not independent replications. The matched pair is the unit of analysis. I used a paired Stuart–Maxwell test to compare the three issue-wording categories. The primary test was recomputed from the reported contingency table (Table 4); the Holm adjustment within a family of nine comparisons and the secondary comparisons are taken from the archived analysis (Section S5). The comparison is observational; its other limitations are discussed in Section 7.

4.6 Retrospective and Information-Guided Replays

The RQ3 replays are a retrospective diagnostic, separate from the RQ2 pairs (definitions and protocols in Section S6). Fourteen historical tasks were assigned three input conditions. O supplies the original issue, W adds its historical CodeQL warning, and P adds a security property derived from that warning without naming the rule. W and P were constructed using historical warnings. The experiment therefore does not establish whether equivalent guidance could be identified before implementing a new task. Of 42 intended outputs, 40 are retained (12 O, 14 W and 14 P), giving 12 complete O/W/P triples; the two missing O outputs are not imputed. One output was retained per available task and input condition. The records link these outputs to scan results and baseline identifiers, but do not establish comparable, independent model sessions or functional equivalence across conditions. The tasks were selected for prior CodeQL warnings rather than sampled randomly.

Section S6 explains why a separate, incompletely documented pilot is excluded from the replay comparisons.

New CodeQL alerts, including those sharing the historical rule, are reported for the 12 complete triples against their recorded baselines (Table 5).

5 Results

The results first follow findings back to the changes that produced them and to the requests those changes served (RQ1), and then turn to issue wording (RQ2) and replay outcomes (RQ3). The main observation is that warnings were not confined to changes unrelated to the requested task. In several examined cases they concerned code that directly addressed the request, where the issue had left the flagged implementation detail unspecified. Other findings lay in tests or documentation and did not by themselves establish exposure in deployed software.

5.1 RQ1 — Findings and the Changes Behind Them

In the first 122 finding-positive instances annotated, 605 scanner findings were linked to 322 responsible changes. The two totals differ because a single change can generate several reports. The spec-workflow-mcp PR is the clearest example. It refreshed a lockfile while addressing a path-translation problem, and its 131 dependency-related findings map to 36 changes, each a package version absent at the merge base and present at the head. Figure 4 shows how far a handful of such instances dominate the finding totals; detailed counts are in Table S5.

Table 3. Findings and responsible changes by finding family and task role.

	Findings	Responsible changes (task role)			
Finding family	Reported	**Total**	Direct	Supporting	Unclear
Source-code warnings	294	**166**	82	82	2
Dependency advisories and downgrades	290	**137**	4	132	1
Secret-like findings	18	**16**	3	13	0
Other dependency-version changes	3	**3**	2	1	0
Total	605	**322**	91	228	3

Original 122 finding-positive instances. A finding is one deduplicated tool report; a responsible change is the changed operation to which one or more findings were mapped, and task role is coded per responsible change. Not the extended 455-change population (Section S4).

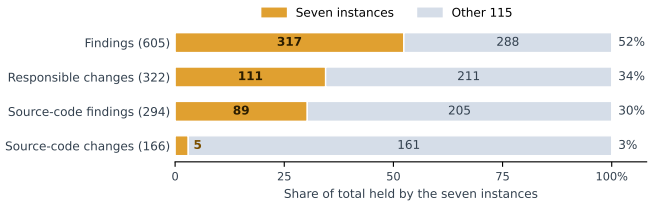


Fig. 4. A few instances hold many findings but almost none of the source-code changes. Each bar splits one total between the seven instances with the most findings and the other 115 in the original cohort. The seven hold 52% of all findings but only 3% of source-code responsible changes. Counts for the seven are the totals minus the excluded-seven counts in Table S5.

Seven instances account for more than half of the scanner reports but only five of the 166 mapped source-code changes. Much of this concentration reflects dependency findings and repeated data-flow reports. Ranking PRs by warning count would therefore foreground a few dependency updates and test operations, and would give a different picture from examining the underlying source-code changes.

The relationship to the requested task also varies by finding family (Table 3). Task role is assigned to each responsible change and is not inferred from the number of warnings it produced. Dependency-related changes were almost all Supporting (132 of 137 for advisories and downgrades), as expected when package versions are resolved alongside the requested work. Source-code changes, by contrast, were as often Direct as Supporting, so about half of the source-code changes behind warnings implemented requested functionality.

The even split between Direct and Supporting source-code changes describes the initial cohort, not a stable population-level balance. Excluding instances with only issue-side agent markers leaves 66 Direct, 79 Supporting and two Unclear changes (Section S4).

A scanner's reported location need not identify the responsible PR change. In the original cohort, data-flow reports often pointed outside the modified test code, sometimes producing many reports from a few changed operations. The detailed location-mapping and sensitivity counts are retained in Section S4.

The affected file also matters. In Paperclip, file-system warnings arose in added regression tests; in dot-ai, a Gitleaks finding concerned a token placeholder in documentation. Neither finding

alone establishes exposure in deployed code (source-linked records in Section S3). Figure S1 shows task-role classifications by reporting tool in the extended mapped set.

5.2 RQ1 — Requested Behaviour and Implementation Choices

A Direct role says that an operation implements what the issue asked for; it does not say that the issue chose how. Three cases make this difference concrete. In each, the requested functionality was delivered through a security-relevant technique that the issue did not specify, and a plausible alternative can be identified. The cases are explanatory examples, not a prevalence sample. Each card gives the request, the implemented operation, the resulting warning and an alternative approach; additional source-linked case evidence is provided in Section S3.

Case 1 — Spotify login verifier `ZHANGTIANYAO1/teamspeak-music-bot`
Issue #112 PR #118

Request	Add Spotify as an audio source next to existing music services. The issue suggests a solution but says nothing about how the login verifier is built.
What the PR did	Builds the verifier by turning each random byte into a character with a remainder operation over a 66-character alphabet. 256 is not a multiple of 66, so some characters are slightly more likely.
Warning	CodeQL reports biased random values in a cryptographic setting.
Reading	The change implements the requested Spotify service and is classified as Direct, but the issue did not prescribe how to generate the verifier. The character distribution is non-uniform; exploitability was not assessed.
Alternative	The pre-PR repository already used `randomBytes(32).toString("base64url")` for session tokens in `src/data/sessions.ts:56`. This is a local example of random-token encoding without modulo reduction, but it was not evaluated as a drop-in replacement for the Spotify verifier. RFC 7636 describes this encoding method for generating a PKCE verifier [14].

Case 2 — GSD tool-call repair `gsd-build/gsd-2`
Issue #2660 PR #3090

Request	Milestone completion fails when the model writes YAML-style bullet lists into JSON tool arguments. The issue suggests making the JSON format clearer in prompts, templates or schema hints.
What the PR did	For the final tool-call arguments, the PR attempts strict parsing, uses a regular expression to repair bullet lists if that fails, and can fall back to a streaming parser. Other streaming paths are not assumed to follow this exact sequence.
Warning	CodeQL reports a regular expression that can take polynomial time on crafted input.
Reading	The runtime repair addresses the reported parsing failure (Direct). A regular-expression repair was not among the remedies the issue suggested.
Alternative	The remedies the issue itself named, or a repair that scans the text once without a backtracking pattern. The first is observed in the issue, the second is general practice.

Case 3 — Grok CLI file mentions `superagent-ai/grok-cli`
Issue #235 PR #236

Request	Load files referenced with @-mentions into the prompt (Explicit instruction).
What the PR did	Looks up the path's metadata and checks it is not a directory, then reads the file in a separate step. The file can change in between.
Warning	CodeQL reports a potential file-system race between checking and reading a path.
Reading	Direct. The issue did not prescribe a check-then-read sequence.
Alternative	Open the file once and read through that handle. A replay on a different file-access task generated a similar pattern (Section 5.4); this does not establish what the Grok CLI implementer considered.

Across these cases, the change is the kind of work the task called for, and the warning concerns something narrower: the particular technique by which the work was done, which the issue left open. The three techniques are unrelated (character selection, text repair and file access), and so are the grounds for regarding each as open, namely a local precedent in the Spotify repository, remedies named in the GSD issue itself, and a single-handle access pattern for Grok CLI. Four further cases

(Neo.mjs, T3Code, KIRI, Paperclip; Section S3) show related distinctions. The Chaibuilder case shows why task role and mechanism need separate labels. The issue requested Tailwind CSS v3, so changing the dependency constraint addresses the request. The six dependency entries associated with advisory reports were classified as Supporting; this does not establish that the downgrade necessarily produced each entry.

The extended analysis asks the same question of all 342 implementation choices. In the primary classification, 6 choices were publicly prescribed, 103 were mechanically generated and 233 were selected among alternatives (Table S6). Public prescription of the flagged technique was therefore rare, which is consistent with the cases above. Because these labels are model-assisted, and 43 choices changed label between classification versions (Section S4), I treat the counts as supporting evidence. The cases illustrate the implementation mechanisms rather than the frequency of any one explanation.

5.3 RQ2 — Issue Specificity in Matched Instances

Explicit implementation instructions appeared in 63.7% of finding-positive issues and 77.9% of matched controls. The controls thus had a 14.2-percentage-point higher proportion of explicit instructions than the finding-positive issues (approximate 95% confidence interval 5.6–22.8 percentage points). The paired comparison across all three wording categories gave a Holm-adjusted $p = 0.00468$ (Table 4). Finding-positive issues more often suggested a solution without giving explicit instructions (33 versus 9), while Problem only issues were similarly frequent in both groups (36 versus 33). The wording labels measure general implementation specificity, not security guidance. Because the two groups were coded in separate passes, coding differences may also contribute to the contrast. Additional comparisons and sensitivity subsets are in Section S5.

Table 4. Issue wording in 190 matched pairs. Rows give finding-positive issues and columns their matched controls with no newly introduced findings under completed applicable checks.

	Control issue wording			
Positive issue	Explicit instruction	Problem only	Suggests solution	Total
Explicit instruction	98	15	8	121
Problem only	25	10	1	36
Suggests solution	25	8	0	33
Total	148	33	9	190

Positive labels were frozen in earlier annotation stages; control labels were produced later from blinded packets. Stuart–Maxwell $\chi^2(2) = 15.12$, $p = 0.00052$ ($p_{\text{Holm}} = 0.00468$). Matching diagnostics and subset results are in Section S5.

The GSD tool-call case (Case 2) illustrates the distinction. Its finding-positive issue suggested prompt, template or schema remedies and was classified as suggesting a solution. Its same-repository control, issue #2545, specified expected operations and files for a queue command and was classified as an explicit instruction. The tasks differ, so this pair illustrates the coding categories rather than an effect of issue wording.

5.4 RQ3 — Alerts and Implementations in Replays

Table 5. Recorded CodeQL alerts in the O/W/P replays. "All" means newly recorded alerts relative to the archived baseline; "Rule" means alerts sharing the historical rule. Each condition has one archived generation; functional equivalence was not established.

	O — issue only		**W — +warning**		**P — +property**	
Task	All	Rule	All	Rule	All	Rule
Ruler — symlink-safe writes	1	1	1	1	0	0
cc-wf-studio — AI commentary	3	1	1	0	1	0
Grok CLI — voice input	2	2	1	1	2	2
Open Design — BYOK key persistence	6	0	3	0	6	0
Oracle — artifact download	2	1	1	1	2	1
Neo.mjs — Fleet service	2	1	0	0	2	1
Delegate Skills — Windows relay fix	2	0	2	0	0	0
oh-my-claudecode	1	1	1	1	1	1
opencodex	1	0	0	0	1	0
Remotion	1	0	0	0	0	0
OpenAgents	0	0	0	0	0	0
Veritas Kanban	0	0	0	0	0	0
Total, 12 complete triples	**21**	**7**	**10**	**4**	**15**	**5**
Lepton	–	–	0	0	0	0
Trilium	–	–	0	0	0	0
Retained outputs	12 of 14		14 of 14		14 of 14	
Passed syntax check	11 of 12		14 of 14		14 of 14	
Median patch size (bytes)[a]	10,482.5		6,305.5		6,631	
Smaller than paired O[a]	–		9 of 12		9 of 12	

O/W/P results were linked to archived baseline SARIF records; the historical scans were not rerun. The original-issue OpenAgents output (case 16633-O) recorded zero new alerts but failed the syntax check; this check is a recorded parse, not a functional task-acceptance test. A dash means unavailable, not zero. [a]Among the 12 complete triples.

Across the 12 tasks with all three conditions, recorded new CodeQL alerts totalled 21 for the issue alone, 10 with the historical warning and 15 with a warning-derived property (Table 5). These totals compare separately generated patches, one per condition, so they cannot separate the supplied information from ordinary variation between generations. Warning-informed patches were also often smaller, and functional equivalence was not established, so a lower count may reflect less implemented functionality rather than a safer implementation. The results therefore describe differences in scanner outcomes rather than demonstrated security improvement.

Case 4 — cc-wf-studio replay `breaking-brake/cc-wf-studio`
Issue #683 PR #695

Request	An independent commentary session that narrates workflow execution in a chat panel (Explicit instruction).
Historical PR	A watcher checks a log file's size by pathname, then separately opens that path and reads through the resulting file descriptor. CodeQL reported a file-system race.
Replay	O uses the same two separate steps. W and P instead inspect the size and read through a single open file handle.
Reading	The warning-informed outputs used a single-handle access pattern instead of separate size-check/read operations.
Limit	New alerts were 3, 1 and 1, with the historical rule only under O. A reconstructed rerun did not reproduce two archived rule hits, and functional equivalence was not checked.

The cc-wf-studio case shows what the totals cannot. Its outputs differ in an identifiable file-access pattern, a check-then-use sequence similar to the one flagged in Case 3, and not only in the number of alerts. Such a code-level difference is the more informative observation, although with one output per condition it remains a single instance.

6 Discussion

6.1 What Tracing Adds to a Warning

A warning arrives as a rule and a location. Tracing adds the changed operation that accounts for it and what the linked issue asked of that operation, and each addition changes how the warning should be read.

Identifying the operation corrects the impression given by counts. Because one dependency update or one added test line can be reported many times, the 605 findings in the original cohort correspond to 322 responsible changes, and the instances with the most findings contain few of the source-code changes. Alert totals partly reflect how tools enumerate a single change, not the number of distinct operations that merit review.

Comparing the operation with the issue separates situations that a list of warnings presents identically. Some flagged operations accompany the requested work, as with regenerated dependency entries or the Paperclip regression tests, and their findings should be assessed in that context rather than as properties of deployed code. Others are the requested work. In the Spotify, GSD and Grok CLI cases, the flagged operation implements what the issue asked for, yet the issue is silent on the flagged detail, whether that is how a verifier's characters are drawn, how malformed arguments are repaired, or whether a file is checked and read in one step or two. Task relevance and security are therefore separate questions. A change can answer the request closely and still warrant security review, because the review concerns a decision that the request never addressed.

This reading differs from the one suggested by controlled attack studies, in which crafted issue text induces a vulnerable patch that still passes tests [2, 11]. There the issue causes the weakness by construction. In the naturally occurring cases examined here, the issues do not point toward the flagged technique; the issue states a goal and the technique enters at implementation. This does not show that public issues are benign in general, since the study did not look for attacks and static analysis can miss induced vulnerabilities [2, 11]. It does indicate that, for interpreting an ordinary warning, the more informative question is which implementation details the request left open rather than what the request instructed.

6.2 What Could Explain a Mechanism Choice

For each of these open details, the public record offers context but not a decision history. Repository helpers, generated dependency files and remedies suggested in an issue are observable before the

change. The pre-PR Spotify repository already encoded session tokens without modulo reduction, and the GSD issue itself proposed prompt and schema changes rather than a runtime repair. Such evidence supports a distinction between a technique that public evidence required, a technique for which a precedent or alternative was available, and a technique produced mechanically by tooling. In the extended model-assisted classification, public prescription was rare (6 of 342 choices), while most choices were selected among alternatives (233) or produced mechanically (103). The evidence does not show that a contributor saw a precedent, considered an alternative or chose deliberately (Section 4.4), and the alternatives named in the cases were not all tested as equivalent replacements. The study therefore characterises what was left open and what was publicly available, not why a particular technique was used.

6.3 Implications and Follow-up

The clearest implication concerns how findings on agent-associated PRs are read. Identifying the responsible change, and noting whether it lies in deployed source, a test, documentation or a lockfile, showed in this cohort which reports repeated a single operation and which operations carried the requested behaviour. The study did not measure whether this information helps reviewers triage warnings. It does show that the same list of alerts supports quite different conclusions once these facts are known.

The same observation bears on measurement. The number of findings is a natural outcome for comparing patch sources or prompts, but in this cohort it was dominated by a few instances and by repeated reports of single changes, so it is best interpreted alongside the changes themselves. RQ3 illustrates the point. Alert totals were lower under warning-informed inputs, yet the more informative difference in cc-wf-studio was a specific file-access pattern.

More tentatively, the details left open in the cases suggest where guidance might matter. The RQ2 association between less explicit issues and new findings is compatible with this view, although it concerns general implementation specificity rather than the flagged technique, the tasks within a pair differ, and the two groups were coded in separate passes. An Explicit instruction label, moreover, did not prevent the Grok CLI issue from leaving its file-access sequence open. Whether stating a security-relevant detail in advance changes the implementation is a question that the present data raise but cannot answer.

Follow-up work can address the limits of the secondary analyses directly. For RQ2, blinded, mixed-order coding of positive and control issues with independent agreement assessment would address the possible coding-pass confounder. For RQ3, a test of guidance should record complete model inputs, repeat each input condition independently and validate that the resulting patches complete the task, so that differences in alerts can be separated from generation variability and from omitted functionality. Warning-derived guidance should remain explicitly retrospective; guidance inferred only from the task and the pre-change repository would test a different, prospective claim. A human-authored comparison could then examine whether the observed patterns are specific to agent-associated work.

7 Threats to Validity

Attribution and annotation. Finding-to-change links and many task-role and mechanism-origin labels were proposed by a language model and retained under the stated evidence rules (Section 4.3). The original workbook documents 11 task-role corrections but does not establish complete, independent human review of the 605 links or a second-annotator agreement rate. These model-assisted annotations should not be treated as independent human ground truth. Errors in judging whether an operation directly fulfils an issue requirement could move changes between Direct and Supporting. No direction of systematic error is assumed. The role distribution also changes when weaker issue-side-only agent markers are excluded (Section 5.1). Results about task roles, concentration and implementation choices therefore depend on these annotations. Four of the 122 original PRs

predate their linked issue, and 15 more were opened within ten minutes afterward. Together with uncertain agent association, this limits any claim about what instructions produced the code.

Sampling and measurement. The corpus is selected using public agent markers, merge status, issue links, language and repository popularity, not verified line-by-line agent authorship. The original 122 positives form an initial annotation batch, not a random sample; no human-authored comparison is available. One dependency change can generate many advisory reports, which drives the concentration result. For 159 findings supported by changed data-flow locations, the archived before/after comparison cannot by itself exclude a previously existing flagged statement whose flow changed. A scanner warning is not a confirmed exploitable vulnerability, and a scan without new findings is not proof of secure code. Test fixtures, documentation and lockfiles also require a different assessment from deployed source.

Matching and replay. The paired wording statistic is reproducible from Table 4, but matching does not control task content or all confounders. Positive and control wording labels were produced in different annotation passes without a cross-pass agreement estimate. Stronger merge-base dependency verification is evidenced for the original annotation cohort only; the comparison therefore retains possible measurement differences. The observed wording difference is an association, not an effect. RQ3 used retrospectively selected tasks and hindsight-derived inputs. With one retained output per task and input condition, the replay analysis cannot distinguish differences associated with the supplied information from variation between model generations. The alert counts are consequently descriptive, not estimates of an intervention effect. The historical alert totals were not independently regenerated, and generated patches were not functionally validated; fewer alerts therefore do not establish prevention or repair.

8 Conclusion

This study traced recorded security findings in merged, agent-associated pull requests back to the changed operations responsible for them and to the requirements of the linked public issues. Doing so shows what warning counts conceal. Findings and changes are different quantities. In the first 122 annotated instances, 605 findings corresponded to 322 responsible changes, and seven instances, dominated by dependency updates and repeated data-flow reports, held more than half of the findings. Most responsible changes in that cohort supported the requested work rather than implementing it, and findings in tests, documentation or dependency files require a different interpretation from findings in deployed source. Where a flagged change did implement the requested behaviour, as in the Spotify, GSD and Grok CLI cases, the warning concerned a security-relevant technique that the issue had not prescribed.

The secondary analyses approach the same question from the side of guidance, with weaker evidence. Issues linked to PRs with new findings less often contained explicit implementation instructions than their matched controls, an observational association whose two groups were coded in separate passes. Retrospective replays recorded fewer CodeQL alerts and, in one examined case, a different file-access pattern under warning-informed inputs, without independent repetitions or verified functionality. Both observations are compatible with flagged techniques entering where a request leaves implementation detail open, and neither establishes a preventive effect. What the study establishes is the value of the intermediate step. A security warning on an agent-associated change becomes interpretable once it is attached to the operation that produced it and to what the issue asked of that operation, because only then can a reader distinguish requested behaviour from accompanying work, and a prescribed technique from one that the request left open. These distinctions give a warning context for review that alert totals alone cannot provide.

Data and Code Availability

The study data and code can be found in the project repository.

References

[1] Pavel Avgustinov, Oege de Moor, Michael Peyton Jones, and Max Schäfer. 2016. QL: Object-oriented Queries on Relational Data. In *30th European Conference on Object-Oriented Programming (ECOOP 2016)*, Leibniz International Proceedings in Informatics, Vol. 56. Schloss Dagstuhl–Leibniz-Zentrum für Informatik, Dagstuhl, Germany, Article 2, 2:1–2:26. doi:10.4230/LIPIcs.ECOOP.2016.2

[2] Simin Chen, Yixin He, Suman Jana, and Baishakhi Ray. 2025. Red Teaming Program Repair Agents: When Correct Patches can Hide Vulnerabilities. arXiv:2509.25894v1. https://arxiv.org/abs/2509.25894v1

[3] Edoardo Debenedetti, Jie Zhang, Mislav Balunović, Luca Beurer-Kellner, Marc Fischer, and Florian Tramèr. 2024. AgentDojo: A Dynamic Environment to Evaluate Prompt Injection Attacks and Defenses for LLM Agents. In *Advances in Neural Information Processing Systems*, Vol. 37. Curran Associates, Inc., 82895–82920. doi:10.52202/079017-2636

[4] Cleidson R. B. de Souza, Emilie Ma, Jesse Wong, Dongwook Yoon, and Ivan Beschastnikh. 2024. Revealing Software Development Work Patterns with PR-Issue Graph Topologies. *Proceedings of the ACM on Software Engineering* 1, FSE, Article 106 (July 2024), 22 pages. doi:10.1145/3660813

[5] Pengfei Gao, Zhao Tian, Xiangxin Meng, Xinchen Wang, Ruida Hu, Yuanan Xiao, Yizhou Liu, Zhao Zhang, Junjie Chen, Cuiyun Gao, Yun Lin, Yingfei Xiong, Chao Peng, and Xia Liu. 2025. Trae Agent: An LLM-based Agent for Software Engineering with Test-time Scaling. arXiv:2507.23370. doi:10.48550/arXiv.2507.23370

[6] Carlos E. Jimenez, John Yang, Alexander Wettig, Shunyu Yao, Kexin Pei, Ofir Press, and Karthik Narasimhan. 2024. SWE-bench: Can Language Models Resolve Real-World GitHub Issues? In *The Twelfth International Conference on Learning Representations (ICLR 2024)*. https://arxiv.org/abs/2310.06770

[7] Brittany Johnson, Yoonki Song, Emerson Murphy-Hill, and Robert Bowdidge. 2013. Why Don't Software Developers Use Static Analysis Tools to Find Bugs? In *Proceedings of the 2013 International Conference on Software Engineering (ICSE '13)*. IEEE Press, Piscataway, NJ, USA, 672–681. doi:10.1109/ICSE.2013.6606613

[8] Hao Li, Haoxiang Zhang, and Ahmed E. Hassan. 2025. The Rise of AI Teammates in Software Engineering (SE) 3.0: How Autonomous Coding Agents Are Reshaping Software Engineering. arXiv:2507.15003. doi:10.48550/arXiv.2507.15003

[9] Yue Liu, Yanjie Zhao, Yunbo Lyu, Ting Zhang, Haoyu Wang, and David Lo. 2025. "Your AI, My Shell": Demystifying Prompt Injection Attacks on Agentic AI Coding Editors. arXiv:2509.22040. doi:10.48550/arXiv.2509.22040

[10] Hammond Pearce, Baleegh Ahmad, Benjamin Tan, Brendan Dolan-Gavitt, and Ramesh Karri. 2022. Asleep at the Keyboard? Assessing the Security of GitHub Copilot's Code Contributions. In *2022 IEEE Symposium on Security and Privacy (SP)*. IEEE, 754–768. doi:10.1109/SP46214.2022.9833571

[11] Piotr Przymus, Andreas Happe, and Jürgen Cito. 2026. Adversarial Bug Reports as a Security Risk in Language Model-Based Automated Program Repair. In *Proceedings of the 23rd International Conference on Mining Software Repositories (MSR '26)*. Association for Computing Machinery, New York, NY, USA, 212–223. doi:10.1145/3793302.3793352

[12] Md Fazle Rabbi, Asif K. Turzo, Arifa I. Champa, and Minhaz F. Zibran. 2026. Insights into Security-Related AI-Generated Pull Requests. arXiv:2604.19965v1. https://arxiv.org/abs/2604.19965v1

[13] Amirali Sajadi, Kostadin Damevski, and Preetha Chatterjee. 2025. Are AI-Generated Fixes Secure? Analyzing LLM and Agent Patches on SWE-bench. arXiv:2507.02976v1. https://arxiv.org/abs/2507.02976v1

[14] Nat Sakimura, John Bradley, and Naveen Agarwal. 2015. Proof Key for Code Exchange by OAuth Public Clients. RFC 7636. Internet Engineering Task Force. https://www.rfc-editor.org/rfc/rfc7636

[15] Tianneng Shi, Kaijie Zhu, Zhun Wang, Yuqi Jia, Will Cai, Weida Liang, Haonan Wang, Hend Alzahrani, Joshua Lu, Kenji Kawaguchi, Basel Alomair, Xuandong Zhao, William Yang Wang, Neil Gong, Wenbo Guo, and Dawn Song. 2025. PromptArmor: Simple yet Effective Prompt Injection Defenses. arXiv:2507.15219. doi:10.48550/arXiv.2507.15219

[16] Xin Tan, Minghui Zhou, and Zeyu Sun. 2020. A First Look at Good First Issues on GitHub. In *Proceedings of the 28th ACM Joint Meeting on European Software Engineering Conference and Symposium on the Foundations of Software Engineering (ESEC/FSE '20)*. Association for Computing Machinery, New York, NY, USA, 398–409. doi:10.1145/3368089.3409746

[17] Peiran Wang, Yang Liu, Yunfei Lu, Yifeng Cai, Hongbo Chen, Qingyou Yang, Jie Zhang, Jue Hong, and Ye Wu. 2025. AgentArmor: Enforcing Program Analysis on Agent Runtime Trace to Defend Against Prompt Injection. arXiv:2508.01249v1. https://arxiv.org/abs/2508.01249v1

[18] Xingyao Wang, Boxuan Li, Yufan Song, Frank F. Xu, Xiangru Tang, Mingchen Zhuge, Jiayi Pan, Yueqi Song, Bowen Li, Jaskirat Singh, Hoang H. Tran, Fuqiang Li, Ren Ma, Mingzhang Zheng, Bill Qian, Yanjun Shao, Niklas Muennighoff, Yizhe Zhang, Binyuan Hui, Junyang Lin, Robert Brennan, Hao Peng, Heng Ji, and Graham Neubig. 2025. OpenHands: An Open Platform for AI Software Developers as Generalist Agents. In *The Thirteenth International Conference on Learning Representations (ICLR 2025)*. https://arxiv.org/abs/2407.16741

[19] Miku Watanabe, Hao Li, Yutaro Kashiwa, Brittany Reid, Hajimu Iida, and Ahmed E. Hassan. 2026. On the Use of Agentic Coding: An Empirical Study of Pull Requests on GitHub. *ACM Transactions on Software Engineering and Methodology* 35, 10 (2026), 1–24. doi:10.1145/3798166

[20] John Yang, Carlos E. Jimenez, Alexander Wettig, Kilian Lieret, Shunyu Yao, Karthik Narasimhan, and Ofir Press. 2024. SWE-agent: Agent-Computer Interfaces Enable Automated Software Engineering. In *Advances in Neural Information Processing Systems*, Vol. 37. Curran Associates, Inc., 50528–50652. doi:10.52202/079017-1601

[21] Zhou Yang, Chenyu Wang, Jieke Shi, Thong Hoang, Pavneet Kochhar, Qinghua Lu, Zhenchang Xing, and David Lo. 2023. What Do Users Ask in Open-Source AI Repositories? An Empirical Study of GitHub Issues. In *2023 IEEE/ACM 20th International Conference on Mining Software Repositories (MSR)*. IEEE, 79–91. doi:10.1109/MSR59073.2023.00024

[22] Qiusi Zhan, Zhixiang Liang, Zifan Ying, and Daniel Kang. 2024. InjecAgent: Benchmarking Indirect Prompt Injections in Tool-Integrated Large Language Model Agents. In *Findings of the Association for Computational Linguistics: ACL 2024*. Association for Computational Linguistics, Bangkok, Thailand, 10471–10506. doi:10.18653/v1/2024.findings-acl.624

[23] Yuntong Zhang, Haifeng Ruan, Zhiyu Fan, and Abhik Roychoudhury. 2024. AutoCodeRover: Autonomous Program Improvement. In *Proceedings of the 33rd ACM SIGSOFT International Symposium on Software Testing and Analysis (ISSTA '24)*. Association for Computing Machinery, New York, NY, USA, 1592–1604. doi:10.1145/3650212.3680384

Supplementary Material

S1 Study Population and Measurement

Selection protocol (GitHub issue–PR corpus)

Query	`is:pr is:merged linked:issue`
Merge window	1 Nov 2025–11 Aug 2026
Repositories	JavaScript/TypeScript; not archived; at least 100 GitHub stars (a sampling filter, not a quality measure)
Agent signal	Public PR-side or issue-side tool marker (issue-side only for 236 instances). This is evidence of association, not proof of agent authorship.

Table S1. Descriptive characteristics of the 3,809 selected issue–PR instances in 723 repositories. Categorical counts and continuous distributions are presented separately.

Categorical measure	**Instances, *n* (%)**
Repository composition	
Instances in TypeScript-primary repositories	3,200 (84.0%)
Instances in JavaScript-primary repositories	609 (16.0%)
Instances from ten largest repositories	700 (18.4%)
Public issue content	
Contains a code block	1,633 (42.9%)
Mentions a file path	1,401 (36.8%)
Includes a log or traceback	471 (12.4%)
Suggests an implementation fix	866 (22.7%)
Pull-request artifacts	
Has recorded review comments	1,194 (31.3%)
Changes test files	2,797 (73.4%)
Changes dependency files	472 (12.4%)
Changes documentation	1,361 (35.7%)

Continuous measure	**Median**	**IQR**	**P_{90}**
Issue body length (words)	200	87–384	663
Lines changed (added + deleted)	160	55–465	1,190
Files changed	4	2–9	19
Commits	2	1–4	8
PR creation to merge (hours)	2.86	0.43–27.01	151.78

Measurement notes. Counts use 3,809 as the denominator; flags can overlap. IQR = 25th–75th percentile; P_{90} = 90th percentile. Repository language is GitHub's primary language, not necessarily the language of every changed file. Features are derived from public metadata; review comments do not establish formal code review, and merge duration is not agent runtime. Instances within a repository are not independent.

Table S2. Security checks, scanner versions and configuration, and PR-attribution criteria. The versions refer to identified runs, not necessarily every scan.

Check	Recorded configuration	Evidence used to attribute findings to the PR
CodeQL	CLI 2.25.6 in a V6 replay; JavaScript query pack 2.3.11 in historical scanner logs. The replay used the JavaScript security-extended suite with build mode none.	Rule, primary location and, where available, data-flow steps in before/after results. In the initial cohort, the retained mapping points to a PR-added primary or flow line; an added flow step does not establish that the flagged statement itself is new.
Semgrep	`p/security-audit` rule configuration; executable version not recovered.	Rule and source location in the before/after comparison, linked to a PR-added line.
OSV-Scanner	Dependency snapshot scans at base and head; executable version not recovered.	Advisory and package/version evidence for a newly introduced dependency state.
Trivy	CLI 0.72.0 in historical run metadata. An observed vulnerability database used format 2 and was updated on 22 July 2026 (UTC).	Advisory and package/version evidence compared across base and head.
Dependency-downgrade analysis	Internal package/version comparison, not a separately versioned scanner.	An older declared or resolved version recorded in a changed dependency hunk.
Gitleaks	Added-content secret detection; executable version not recovered.	Rule and secret-like match on PR-added content; placeholder or test strings were not automatically excluded.

Attribution and coverage. Original-cohort merge-base verification supports 142 retained dependency links. Applicability, execution and findings were recorded separately. A failed, unavailable or inapplicable check is not a negative result. Recorded version observations do not establish the complete executable and advisory data used for every historical run. The annotation instructions and scanner configuration accompany the study code.

S2 Classification and Attribution

Codebook cross-reference

Task role and implementation-choice categories are defined in Table 2. Table S3 gives the additional evidence tiers for M choices.

Table S3. Explanation tiers, applied to M choices only.

Tier	Meaning
T1 direct reuse	An earlier, near-identical mechanism was reused. Public precedent is not proof that it was read.
T2 strong precedent	A materially equivalent helper, architecture or convention exists.
T3 explicit guidance	A pre-change contract or instruction constrains the mechanism without prescribing it (not H).
T4 contextual	The task, API or local style makes the mechanism understandable without choosing it.
T5 none found	No independent public explanation after a sufficiently complete review. Not the same as autonomous invention.
Incomplete	Material evidence is missing or the chronology is uncertain; this prohibits T5.

Protocol A — Reconstructing and mapping findings (original 122 instances)

01 **Assemble / deduplicate.** Collect issue, PR, diff and tool evidence; merge duplicate records of the same finding within an instance, not distinct rules or advisories (832 raw references → 605 findings).

02 **Propose responsible operation.** Give the language model the finding, tool evidence and PR diff, but not the issue or PR description. Record its proposed finding-producing code or dependency operation.

03 **Verify dependency transitions.** At the true merge base and PR head, check 987 candidate package/version transitions, of which 274 were introduced, 697 were already present and 16 were not reproduced. These checks excluded 12 initially positive instances after four earlier merge-base exclusions, supported 142 retained dependency links, and resolved 36 previously unresolved links, all associated with case 13918. Individual link-basis fields do not distinguish those 36. Comparable coverage is not established for the 70 later positives. These checks address PR attribution, not advisory validity.

04 **Check supporting code evidence.** Retain a link only with an identified changed file, changed line/hunk and explanation connecting the change to the finding; otherwise leave unresolved. All 605 mapped findings passed the recorded gate (Table S4).

Protocol B — Validation and a separate blind description pass

05 **Validation boundary.** Each retained link has recorded supporting evidence, but the workbook does not demonstrate complete independent human re-annotation.

06 **Blind patch descriptions.** Model-assisted descriptions of 134 candidate diffs omitted issue wording, PR goal and scanner outcome. A second pass left 133 descriptions unchanged and corrected one. Subsequent dependency checks excluded 12 cases, leaving 122 active cases. These descriptions neither determined the mappings or task roles nor constitute a human agreement study.

Blinded evaluation inputs and preserved patch-summary prompt

Exact preserved Stage 4B instruction. *"Describe the substantive implementation changes made by this patch. Be factual, do not infer motivation, security impact, or correctness. Summarize the main changes in one sentence."*

Issue summary	Issue title/body only; a factual sentence with a verbatim source excerpt, before PR or scanner evidence.
PR summary	PR title/body only; the stated goal was summarised without the linked issue or code.
Patch summary	Diff and changed-file metadata only; issue, PR-description and scanner results excluded.
Task-role label	Frozen issue description and mapped responsible change; scanner rule, severity and PR goal excluded.
Choice classification	Public evidence available before the change was used to distinguish prescribed, mechanical, selected and unresolved mechanisms.

The annotation instructions and scanner configuration are provided with the study code.

Table S4. Mapping evidence for the 605 findings of the original 122-instance cohort.

Mapping basis	Findings	What it establishes
Primary location on a PR-added line	135	The flagged statement was added by the PR
Data-flow or related location on a PR-added line	159	An added line lies on the reported flow; the flagged statement may predate the PR (Ruler, Open Design, GSD tests)
Changed version hunk or merge-base verification	293	The package version is introduced by the PR's dependency change
Added-content anchor (Gitleaks)	18	The matched string was added by the PR
Total	605	

S3 RQ1 Case Evidence

Case-card key and coverage

Coverage	Spotify, GSD repair, Grok CLI file mentions and cc-wf-studio appear in the main text; further examples follow below.
Issue labels	Problem only; Suggests solution; Explicit instruction.
Supporting	Corresponds to the original workbook label "Side-effect".
Traceability	GitHub links point to recorded commits; case identifiers refer to the original 122-instance workbook unless otherwise stated.

spec-workflow-mcp `Pimzino/spec-workflow-mcp` Case 13918

Issue #172 PR #173 approvals.ts

Request	Fix approvals-tool failures in `PathUtils.translatePath` for relative, absolute and space-containing paths (Problem only).
What the PR did	Added a defensive translation wrapper and path tests, and refreshed `package-lock.json`.
Warning	131 dependency advisories on lockfile packages (e.g. `@babel/core` 7.28.5, `ajv` 8.17.1).
Reading	36 changes, each a package version absent at the true merge base and present at the head (supported by merge-base verification). All Supporting.
Limit	One lockfile refresh, not 131 decisions, and one of the seven instances holding 317 of 605 findings. Reachability of any advisory is not established.

GSD shutdown and recovery tests `gsd-build/gsd-2` Cases 290 and 291

Issue #1797 PR #1821 Issue #1808 PR #1819 signal-handlers.test.ts doctor-completion-deferral.test.ts

Request	The first issue requests handling SIGTERM, SIGHUP and SIGINT and cleaning crash and session locks (Explicit instruction). The second requests recovery when a session stops between the last task and slice-completion dispatch (Suggests solution).
What the PRs did	The signal-handler test registers a handler on `/tmp/test-signal-handlers` (line 24). The deferral test builds a directory from `tmpdir()`, `Date.now()` and `Math.random()` (line 20).
Warning	CodeQL reports 42 and 44 findings for `js/insecure-temporary-file`, reported in other files (`captures.ts`, `export.ts`, `auto-recovery.ts`).
Reading	Two test operations account for all 86 findings, mapped through a data-flow location on a PR-added test line. Supporting. The second PR has three more single-finding test changes.
Limit	Whether the 86 are newly reachable or existing sinks is not established, because a newly added flow step does not independently establish a newly introduced flagged sink (Table S2). Nor is exploitability.

Chaibuilder Tailwind downgrade `chaibuilder/core` Case 15703

Issue #610 PR #611 package.json

Request	Change the Next.js framework from Tailwind CSS v4 to v3 because the builder supports v3 (Explicit instruction).
What the PR did	Changed the `tailwindcss` dependency constraint from `^4` to `^3.4.19` in `frameworks/nextjs/package.json`:32, adjusted build scripts and refreshed `pnpm-lock.yaml`.
Warning	15 findings comprising one dependency downgrade and 14 advisories on lockfile entries (`ajv`, `dompurify`, `fast-uri`, `linkify-it`, `lodash`, `markdown-it`).
Reading	Seven changes. The dependency-constraint change is Direct and forms one mechanism-selection choice, because the issue requests v3 but does not specify the dependency constraint. Six associated lockfile entries are Supporting and classified as mechanically produced changes.
Limit	Who chose the version, the reachability of any advisory, and whether the downgrade is a defect are not established.

Neo.mjs wake receiver `neomjs/neo` Case 6150

Issue #16180 PR #16183 receiver.mjs

Request	A signed, durable local wake receiver with restricted manifest state (Explicit instruction).
What the PR did	Checks the manifest with `fs.stat(path)`, then parses `fs.readFile(path)`.
Warning	CodeQL `js/file-system-race`, `receiver.mjs`:48. A second change in the file (`js/user-controlled-bypass`) is not discussed here.
Reading	One change, on a PR-added line. Direct. An explicit permission check does not remove the check-then-read sequence.
Limit	An exploitable race is not established. Not the Neo.mjs Fleet replay task.

T3Code scoped packages `pingdotgg/t3code` Case 3504

Issue #4142 PR #4167 composerInlineTokens.ts

Request	Keep pasted text such as `yarn expo install @expo/ui` as text, or ask before treating `@expo/ui` as a folder (Suggests solution).
What the PR did	`SCOPED_PACKAGE_REFERENCE_REGEX` recognises unquoted `@scope/package` references and exempts them from file-mention extraction.
Warning	CodeQL `js/redos` (line 28) and `js/polynomial-redos` (line 66).
Reading	Two findings, two changes on PR-added lines, both Direct. They concern one recognition strategy, not two decisions.
Limit	The study did not demonstrate a slowdown.

KIRI PHP tag detection `CAPHTECH/kiri` Case 154

Issue #25 PR #26 codeintel.ts

Request	PHP symbol extraction, indexing, tests and documentation (Explicit instruction).
What the PR did	`phpTagRegex` inspects the start of a file, allowing whitespace, a BOM or a shebang before the PHP opening tag.
Warning	CodeQL `js/redos`, `codeintel.ts`:177.
Reading	The changed expression appears on a PR-added line and implements the requested PHP-symbol feature (Direct), although the issue did not prescribe that expression.
Limit	The study did not demonstrate a slowdown.

NotionNext originality proof `notionnext-org/NotionNext`

Issue #4278 PR #4348 later PR #4353 originalityProof.js originalityProofManifest.js

Request	A content-origin (originality) proof with trusted timestamp metadata.
What the PRs did	PR #4348 adds an in-memory proof and hash. PR #4353 reads the on-disk manifest, compares contents and writes an updated manifest by pathname.
Warning	CodeQL `js/file-system-race`, `originalityProofManifest.js`:98, archived for PR #4353 only.
Reading	Extended provenance records contain a mapping for the later PR, but its individual fields have not been independently revalidated for this case card. The case is outside the original 122-instance workbook.
Limit	Successive PRs on one request moved the mechanism from memory to persistence. The finding must not be attributed to PR #4348, and the two PRs are not a task-equivalent comparison.

Paperclip test warnings `paperclipai/paperclip` Case 4408

Issue #5028 PR #5240 codex-home.test.ts

Request	Keep managed Codex authentication synchronised so stale copied refresh tokens do not break runs (Suggests solution).
What the PR did	Replaces a stale `auth.json` copy with a symlink. Regression tests create and read temporary `auth.json` files.
Warning	Two CodeQL `js/file-system-race` warnings, `codex-home.test.ts`:99 and :121.
Reading	Two changes on PR-added lines. Supporting (test).
Limit	Both warnings lie in test code. Production exposure is not established.

dot-ai README placeholder `vfarcic/dot-ai` Case 6017

Issue #281 PR #286 README.md

Request	Optional Gateway API support with Helm resources, configuration examples and documentation (Suggests solution).
What the PR did	A README `curl` example sets an Authorization header to the literal `YOUR_AUTH_TOKEN`.
Warning	Gitleaks `curl-auth-header`, `README.md`:245.
Reading	One mapped change, classified Direct because the issue requested documentation. The affected artifact is documentation. The archived record labels it "production source", but the path is `examples/gateway-api/README.md`. The accompanying correction record identifies this as documentation. Figure S1 restores the original file-type panel with this known discrepancy explicitly noted; no aggregate percentages have been recalculated.
Limit	An instructional placeholder, not a usable credential. Credential exposure is not established.

S4 Extended RQ1 Results

Table S5. Findings versus responsible changes with and without the seven highest-count instances (original 122-instance cohort; workbook).

Scope	Findings	changes	Note
All 122 instances	605	322	1.88 findings per change
Excluding the seven instances (115)	288	211	1.36 findings per change
Source-code findings, all / excluding seven	294 / 205	166 / 161	Five code changes lie in the seven (two GSD instances)
Dependency findings, all / excluding seven	293 / 65	–	Advisories, downgrades and other version changes
Seven highest-count instances	317 (52.4%)	–	228 dependency-related; five instances chiefly involve dependency reports, two chiefly data-flow reports
spec-workflow-mcp PR #173	131	36	Lockfile refresh
GSD cases 290/291	86	2	Data-flow reports in other files

Location-mapping and sensitivity checks (original 122 instances)	
Mapping locations	Of 294 source-code findings, 135 were linked through a primary location on an added line and 159 through an added data-flow or related location. The latter cover 37 changes in 32 instances; two GSD test operations account for 86 reports.
Excluding flow-mapped findings	446 findings and 285 changes remain.
Excluding issue-only agent markers	580 findings and 302 changes remain; source-code roles in this subset are 66 Direct, 79 Supporting and two Unclear (147 changes).
Applying both restrictions	428 findings and 268 changes remain.
Interpretation	The finding-to-change distinction persists, although the original 82/82 Direct/Supporting split is sensitive to the restriction. A new data-flow step can cause a query to flag an existing statement, so a mapped report does not necessarily identify a new flagged statement.

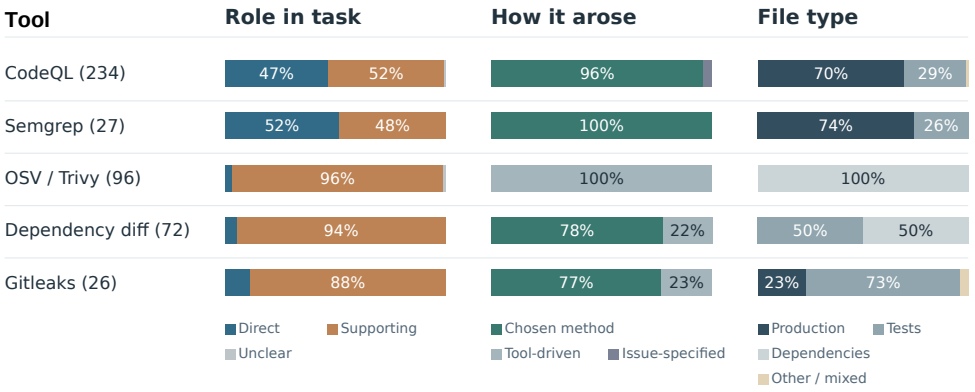


Fig. S1. Tool-level composition of 455 mapped changes in 191 instances. Each row uses the tool's change count as denominator, and each panel has its own legend. Task-role colours denote Direct, Supporting and Unclear. The other panels show the original implementation-origin and file-type labels. These change-level labels are distinct from the 342-choice classification in Table S6, and the version behind the origin panel is uncertain. The file-type panel retains the known dot-ai README misclassification discussed in Section S3; it is not a corrected estimate of production exposure. All original bars and percentages are restored without recalculation.

Table S6. How 342 implementation choices were classified using public evidence (455 responsible changes, 191 instances).

Version	H	T	M	U
Primary frozen classification	6	103	233	0
Earlier classification snapshot	5	103	234	0

Among the 233 choices classified as selected among alternatives, 77 have a specific precursor, precedent or contract (T1–T3), 128 have contextual support alone (T4), four have no independently identified public explanation (T5), and 24 have critical evidence missing (Incomplete). These are the final model-assisted classifications. The preceding classification version differed for 43 implementation choices; it is shown to expose version sensitivity, not pooled into the primary analysis. Independent blind human agreement on these labels is unavailable. Similar aggregate totals do not imply stable individual classifications. "Incomplete" concerns the explanation tier, whereas U concerns the broader H/T/M classification; incomplete evidence for an earlier precedent does not automatically make that broader category unresolved. Zero U labels therefore does not establish complete certainty.

Additional patterns from archived records

`js/insecure-temporary-file`	44 responsible changes in 18 repositories; 26 are test-only Supporting changes.
`js/file-system-race`	17 responsible changes in 13 repositories. One responsible change can be associated with multiple rules.
Tool-generated choices	At least one choice classified as mechanical appeared in 13/20 dependency-maintenance or compatibility/migration instances, versus 2/38 feature instances in the initial subset.

S5 RQ2 Matching and Sensitivity

Matched-control procedure (RQ2)

01	Identify eligible candidate controls. Same-repository candidates must cover the positive's completed checks; cross-repository candidates must have exactly the same set of completed checks.
02	Apply the frozen date and change-size calipers (Section 4.5); exclude ineligible candidate pairs.
03	Solve one-to-one matching without replacement, prioritising maximum pair count, then same-repository pairs, then minimum normalised date/size distance, in that order.
04	Freeze the pair map; only then join the issue-wording annotations. No pair-specific re-annotation. Figure S2 shows the matching stages.

Issue-wording codes and inference boundary

Problem only	Problem or goal without a proposed implementation.
Suggests solution	Approach proposed, without fixing the method.
Explicit instruction	A concrete implementation instruction; distinct from H (exact mechanism prescribed).
Statistical test	Paired 3 × 3 Stuart–Maxwell test with matched pairs as units; categories are not a numeric score. Explicit-versus-other proportions are descriptive.
Multiple tests	Holm correction covered nine comparisons. These assessed issue purpose, instruction specificity, issue–PR goal alignment, implementation category, formal human review, issue-to-PR delay, shared issue/PR authorship, prior requester activity, and coding-agent or tool identity.
Interpretation	Issue-wording specificity is not a measurement of security requirements or the precise risky mechanism.

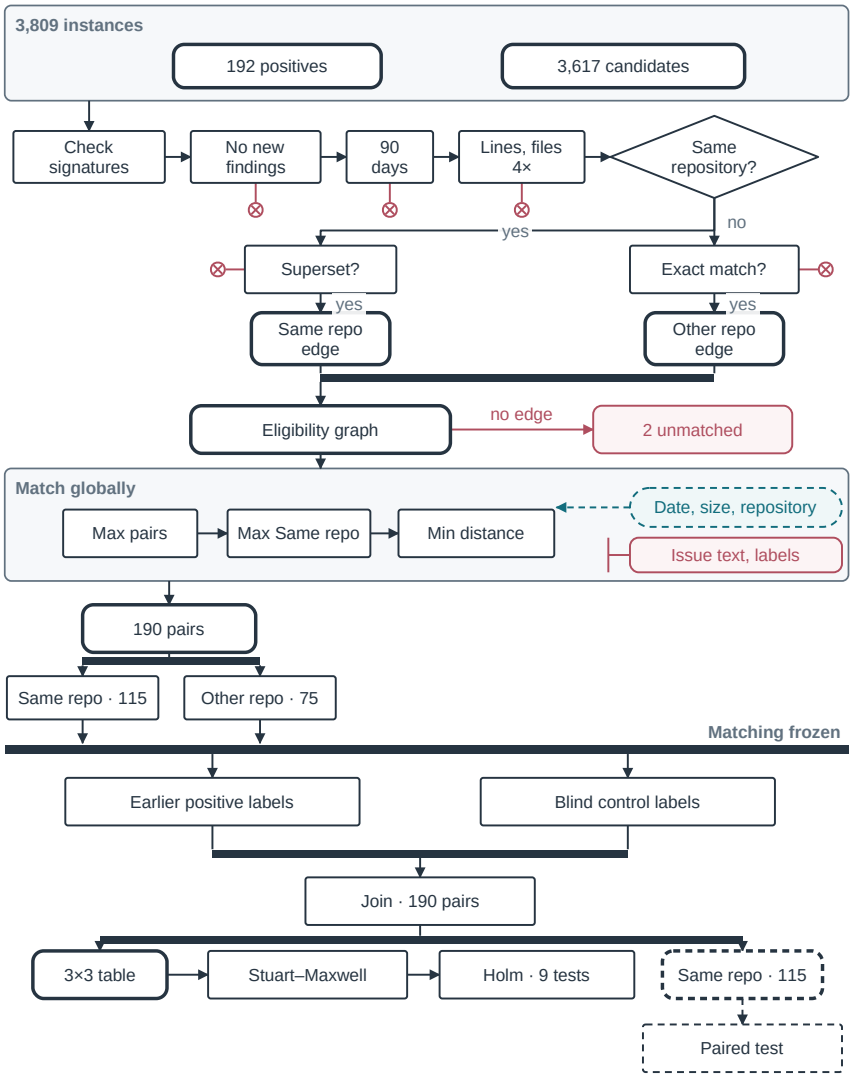


Fig. S2. RQ2 matched comparison, showing eligibility checks and one-to-one matching. Labels were joined to pairs only after matching was frozen. Red end markers indicate failed eligibility checks; the dashed branch shows the nested same-repository sensitivity analysis.

Table S7. RQ2 matching diagnostics, subsets and secondary comparisons. Values come from archived analysis unless identified as recomputed.

Quantity	Value
Matching	
Positives matched	190 of 192 (115 same-repository, 75 cross-repository); two without an eligible control
Calipers	90 days PR date; about fourfold changed lines and files
Median absolute date separation	6.66 days
Median larger-to-smaller ratio	1.26 changed lines; 1.22 changed files
Wording, all 190 pairs (Table 4)	
Explicit instruction	121 positives vs 148 controls (−14.2 points)
Discordant pairs	Explicit 23 positive-only vs 50 control-only; Suggests solution 33 vs 9; Problem only 26 vs 23 (recomputed from Table 4)
Subsets (nested, not independent)	
Original-cohort pairs	73 of 121 vs 93 of 121 explicit (−16.5 points)
Same-repository pairs	77 of 115 vs 95 of 115 explicit; paired $p = 0.00645$
Direct-only positive instances	48 of 78 vs 58 of 78 explicit
Supporting-only positive instances	60 of 94 vs 75 of 94 explicit
Mixed positive instances	13 of 15 on both sides; three pairs unassigned
Secondary comparison	
Issue–PR goal alignment	160 of 190 vs 183 of 190; $p = 0.00102$, $p_{\text{Holm}} = 0.00815$; within repositories $p \approx 0.077$

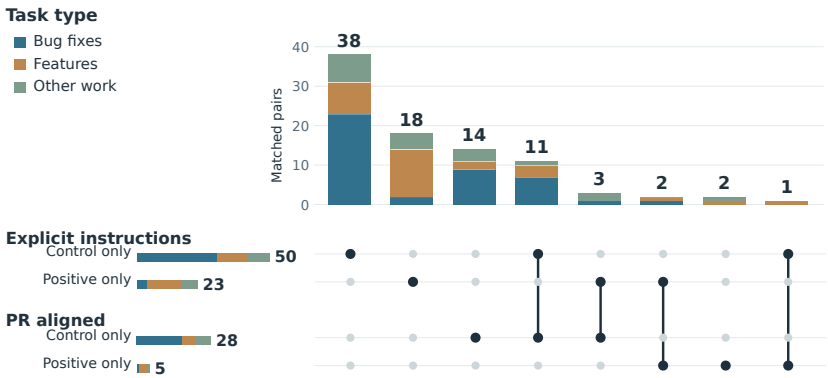


Fig. S3. RQ2 paired differences in issue explicitness and PR goal alignment, by task type of the finding-positive issue. Bars show the 89 of 190 pairs that differ on at least one measure; the other 101 have the same status on both sides. Descriptive, not causal.

S6 RQ3 Replay Evidence

Table S8. Inputs for the retrospective O/W/P comparison. W and P use information from the historical warning; they do not test whether equivalent guidance could be identified prospectively.

Condition	Information supplied	Interpretation
O	Original public issue alone.	Baseline reimplementation.
W	Issue plus its historical CodeQL warning.	Retrospective warning information.
P	Issue plus a warning-derived security property, without naming the rule.	Retrospective property information.

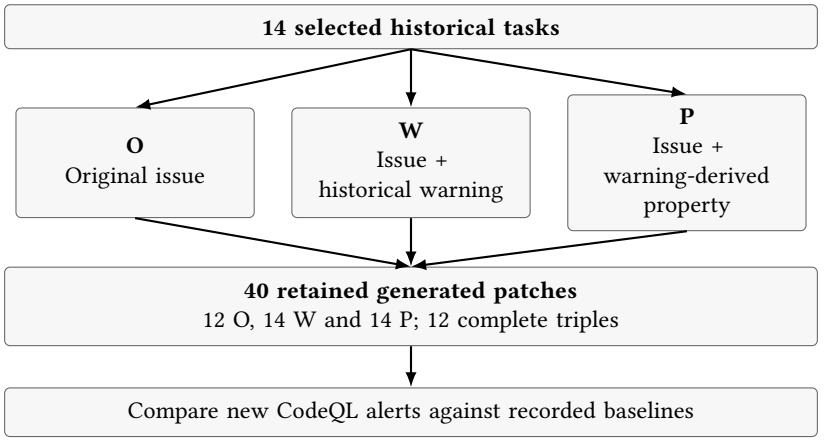


Fig. S4. The O/W/P replay comparison. Each available task-condition combination has one retained output. Independent generation sessions and functional equivalence were not verified, and missing O outputs are not counted as zero.

Replay records and excluded pilot

Section 4.6 describes task selection and the available output-to-scan links. The syntax check covers O/W/P outputs only. The OpenAgents original-issue output (case 16633-O) recorded zero new alerts but failed that check; the check does not establish functional task completion.

A separate P1–P5 guidance pilot is excluded from these comparisons. The available records do not establish whether its summary corresponds to the intended rerun, whether the stages were independent model generations, or which pinned baseline and head scans support every result. Some outputs or intermediate records are also missing. The complete original protocol descriptions, reported counts and qualifications are preserved separately in the source package; they are not evidence of a guidance effect here.

Replay-case coverage

The following six cases supplement cc-wf-studio (main text, Case 4) for the O/W/P comparison in Table 5. Other tasks with complete triples are oh-my-claudecode #257, opencodex #1048, Remotion #10147, OpenAgents #8637 and Veritas Kanban #906. Original-issue outputs are missing for Lepton #226 and Trilium #7571.

Ruler symlink-safe writes `intellectronica/ruler` Case 3429

Issue #570 PR #571 CrushAgent.ts

Request	Generated writes must not follow a symlink at the final output path, including when backups are disabled (Explicit instruction).
Historical PR	Replaces direct writes with `writeGeneratedFile`, which already checks for symlinks.
Warning	CodeQL `js/insecure-temporary-file` at `FileSystemUtils.ts`:345, a statement already in the base commit. Mapped through the flow step `CrushAgent.ts`:84 (Direct).
Replay	O and W each recorded one new alert; P recorded zero.
Limit	Zero-alert outputs are not shown to implement the task or to repair the warning site, which predates the PR.

Grok CLI Telegram voice input `superagent-ai/grok-cli` Case 8152

Issue #208 PR #210 audio-input.ts

Request	Support voice input through Telegram (Problem only).
Historical PR	Fetches media from `api.telegram.org/file/bot. . . /{file_path}`, creates a temporary directory with `mkdtemp`, writes the downloaded audio file with mode `0600`, transcribes it, and removes the temporary directory.
Warning	CodeQL `js/file-access-to-http` (line 71) and `js/http-to-file-access` (line 79). Both Direct, on PR-added lines.
Replay	The O/W/P records contain two, one and two new alerts, respectively.
Limit	The reports concern network/file flows in voice processing. This is a different task from the Grok CLI file-mentions case.

Open Design key storage `nexu-io/open-design` Case 12344

Issue #6330 PR #6356 runs.ts

Request	Persist BYOK keys reliably on Windows and fail clearly instead of silently falling back to cloud billing (Suggests solution).
Historical PR	The PR rolls back the secure credential-profile design to browser-local BYOK configuration. The mapped change is a configuration-completeness check in `routes/runs.ts`:849.
Warning	CodeQL `js/remote-property-injection` and `js/polynomial-redos` at `byok-opencode.ts`:80 and :104, statements present before the PR. One Direct change, mapped through a flow location.
Replay	None of the three O/W/P outputs recorded an alert under either historical rule; their recorded alerts concern other rules.
Limit	Absence of the historical rules did not mean an alert-free output; the recorded alerts concern other rules.

Oracle downloadable artifacts `steipete/oracle`

Issue #244 PR #245 artifacts.ts

Request	Save browser-generated downloadable artifacts rather than only linking them in a transcript.
Historical PR	Writes downloaded bytes using `writeBinaryBrowserArtifact`, which sanitises the filename and checks for an unused target path before writing.
Warning	CodeQL `js/http-to-file-access`. Extended provenance records contain a mapping, not independently revalidated for this case card; this task is outside the original 122-instance workbook.
Replay	The historical rule was recorded under all three O/W/P conditions.
Limit	Functional equivalence and any repair are not established.

Neo.mjs Fleet service `neomjs/neo`

Issue #16735 PR #16761 fleetHealthcheck.mjs

Request	An optional authenticated Fleet server and healthcheck probe with file-sourced bearer credentials.
Historical PR	The probe reads a bearer token from a configured file and sends it to a configurable URL (default loopback, with a timeout).
Warning	CodeQL `js/file-access-to-http`, `fleetHealthcheck.mjs`:107. An extended-provenance mapping is archived but not independently revalidated here; this task is outside the original 122-instance workbook.
Replay	The property condition matched the original-issue condition in recorded alert counts.
Limit	The zero-alert W output may omit functionality. This is a different task from the Neo.mjs wake receiver.

Delegate Skills Windows relay `amElnagdy/delegate-skills`

Issue #3 PR #13 relay.mjs

Request	Fix Windows quoting and path failures in an existing codex-delegate relay.
Historical PR	A new Grok relay writes brief, event and result files beneath an `os.tmpdir()` path named from repository and timestamp, created recursively rather than with `mkdtemp`.
Warning	Four CodeQL `js/insecure-temporary-file` locations in `relay.mjs`. An extended-provenance mapping is archived but not independently revalidated here; this task is outside the original 122-instance workbook.
Limit	The linked issue and PR differ in scope, limiting what a reimplementation of the issue can test.

Wits University Faculty of Science post-graduate student AI declaration

I understand that the use of generative AI tools (such as ChatGPT or similar) without explicitly declaring such use constitutes a form of plagiarism and is classified by Wits University as academic misconduct.

I declare that in the course of conducting the research towards my degree or in the preparation of this thesis/dissertation/research report (select one by marking with an X):

I **did not** make use of generative AI tools ☐

I **did** make use of generative AI tools for the following (tick all that apply):

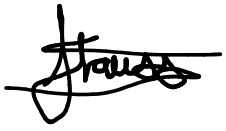
1. Idea Generation (research problem/design, hypothesis) ☑
2. Sourcing Related Work (summarising, identifying sources) ☑
3. Methods and Experiment Design (experiment setup, model tuning) ☐
4. Data Analysis (presentation, coding, interpretation) ☑
5. Theoretical Development (theorem proving, conceptual analysis) ☐
6. Code Development (generating algorithms, writing scripts) ☑
7. Presentation (rendering graphics, formatting) ☐
8. Editing (grammar, readability) ☐
9. Writing (text generation, document structuring) ☑
10. Citation Formatting (structuring, organising) ☑

If other uses were involved, please specify below:

Generative AI tool used (list all)	**Used for?**
NotebookLM	Sourcing Related Work, Writing
Gemini	Idea Generation, Citation Formatting
ChatGPT	Patch Generation, Writing, Arguing

If generative AI tools were used as an integral part of the experimental design or in the direct execution of my research, I confirm that details of this use are clearly outlined in the relevant experimental/methodology chapters of my thesis/dissertation/research report.

Student number: 2613224

Candidate signature: 

Date: 02/11/2026